Reviewer Pack — Riemannian Curvature Tensor Bounds for Communication Complex…

Entry 0e52778f601d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 00:48:25 UTC

Riemannian Curvature Tensor Bounds for Communication Complexity of XOR Functions

Entry ID: `0e52778f601d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 00:48:25 UTC

1. Conjecture

Field A (mathematical branch): Riemannian Geometry **Field B** (complexity object): Communication Complexity

Statement:

> ['For any given XOR function f with n inputs, the Riemannian curvature tensor of a metric space X representing the communication complexity of f is upper-bounded by Ω(√n).', 'This bound holds for all metrics that can be realized on a discrete set with constant precision.', 'Conversely, if there exists an XOR function f such that the Riemannian curvature tensor of its corresponding metric space X is less than Ω(√n), then f can be computed by a communication protocol with complexity O(n).']

Rationale (proposer's reasoning):

> ['Riemannian geometry has been previously applied to study the geometry of data, but not directly to communication complexity. The curvature tensor captures intrinsic geometric properties that might relate to the structure of communication protocols.', 'The relationship between communication complexity and geometric curvature could potentially reveal new insights into the efficiency of information exchange in distributed computing.']

Taxonomy category: `ACC_SIPSER` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9ad9746073ad6679`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The Riemannian curvature tensor for XOR functions is considered supported if the lower bound is Ω(√n) and falsified if any curvature tensor value is less than Ω(√n).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Riemannian curvature tensor" AND "communication complexity" AND XOR - "metric space" AND Riemannian geometry AND bounds ON communication complexity - XOR function computation AND Riemannian metric AND complexity analysis

Top relevant hits considered: - [http://arxiv.org/abs/2501.03128v1] C*-rigidity of bounded geometry metric spaces - [http://arxiv.org/abs/2504.03362v2] Metric spaces with small rough angles and the rectifiability of rough self-contracting curves - [http://arxiv.org/abs/1710.11333v1] Spectral geometries on a compact metric space

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.7s

5.1 Generated Python source

```python
import random
import math

def generate_xor_function(n):
    return [random.choice([0, 1]) for _ in range(2**n)]

def euclidean_distance(x, y):
    return sum((xi - yi) ** 2 for xi, yi in zip(x, y)) ** 0.5

def construct_metric_space(f):
    n = len(f)
    metric_space = {}
    for i in range(2**n):
        x = [int(b) for b in f"{i:0{n}b}"]
        for j in range(i + 1, 2**n):
            y = [int(b) for b in f"{j:0{n}b}"]
            metric_space[(tuple(x), tuple(y))] = euclidean_distance(x, y)
    return metric_space

def riemannian_curvature_tensor(metric_space):
    n = len(next(iter(metric_space)))
    tensor = {}
    for (x1, x2), d1 in metric_space.items():
        for (y1, y2), d2 in metric_space.items():
            if x1 == y1 and x2 == y2:
```

```
                continue
            numerator = 0
            denominator = 0
            for z in range(2**n):
                z_x1 = [int(b) for b in f"{z:0{n}b}"]
                z_y1 = [int(b) for b in f"{z:0{n}b}"]
                d3 = euclidean_distance(z_x1, z_y1)
                numerator += (d1 * d2 - d3**2) / (d1 + d2 - 2 * d3)
                denominator += (d1 + d2 - 2 * d3)**2
            tensor[(x1, x2, y1, y2)] = numerator / denominator
    return tensor

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        f = generate_xor_function(n)
        metric_space = construct_metric_space(f)
        curvature_tensor = riemannian_curvature_tensor(metric_space)

        lower_bound = math.sqrt(n)
        upper_bound = 2 * math.sqrt(n)

        for (x1, x2, y1, y2), value in curvature_tensor.items():
            if value < lower_bound or value > upper_bound:
                return {
                    "metric_name": "Riemannian Curvature Tensor",
                    "metric_value": value,
                    "instances_tested": 1,
                    "conjecture_holds": False,
                    "counterexample": f"Curvature tensor {value} is not in the range [{lower_bound
                }

    return {
        "metric_name": "Riemannian Curvature Tensor",
        "metric_value": None,  # Not applicable for this conjecture
        "instances_tested": len(n_values),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
```

```
        result_message = f"SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}"
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        result_message = f"FALSIFIED counterexample=\"Curvature tensor out of bounds\" first_faili

    print(f"RESULT: {result_message}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which prevents us from verifying whether the Riemannian curvature tensor for XOR functions is less than Ω(√n). | next: Run the test again with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12606
2	propose	ollama_remote	glm4:latest	0	0	10282
3	preregistration	ollama_remote	glm4:latest	0	0	5286
4	novelty	ollama_remote	glm4:latest	0	0	4638
5	novelty	ollama_remote	glm4:latest	0	0	5756
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11440
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12964
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10190
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11177
10	judge	ollama_remote	glm4:latest	0	0	39529

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 123867 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/0e52778f601d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0e52778f601d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0e52778f601d.tar.gz` (if generated)